BLOG

View as Markdown

ClickHouse® and Redshift Face Off Again in NYC Taxi Rides Benchmark

By Alexander Zaitsev • 27th August 2020

Join our Slack

ClickHouse® is a registered trademark of ClickHouse, Inc.; Altinity is not affiliated with or associated with ClickHouse, Inc.

Table of Contents:

ClickHouse is famous for its performance, and benchmarking expert Mark Litwintschik praised it as being "the first time a free, CPU-based database has managed to out-perform a GPU-based database in my benchmarks". Mark uses a popular benchmarking dataset with NYC taxi trips data over multiple years. The current size is 1.3 billion rows. We already used this dataset in our blog 3 years ago, comparing ClickHouse to Amazon Redshift, so it is time to refresh the results.

Setup

We start with the latest ClickHouse version 20.6.6.44 running inside Kubernetes on an Amazon m5.8large EC2 instance. This is a mid-range instance with 32 vCPUs, 128GB of RAM and EBS gp2 storage, that is priced at $1.54 per hour or $36.86 per day in AWS. EBS users also have to pay for storage $3 per terabyte per day.

Data loading

In previous benchmarks using NYC Taxi Rides datasets, users had to go through a painful and lengthy data transformation process that could take hours. Those times are gone, thanks to contributions to ClickHouse by Altinity engineers. We can use new ClickHouse capabilities in order to load data directly from S3 bucket.

The data is stored in 96 gzip-ed CSV files, one file per month, several hundred MBs size each:

Name	Last modified	Size	Storage class
data-200901.csv.gz	Jul 16, 2020 8:00:29 PM GMT+0300	426.1 MB	Standard
data-200902.csv.gz	Jul 16, 2020 8:00:29 PM GMT+0300	407.0 MB	Standard
data-200903.csv.gz	Jul 16, 2020 8:00:29 PM GMT+0300	439.7 MB	Standard

The total size reported by Amazon is 37.3GB:

nyc_taxi_rides/data/tripdata/
96 Objects – 37.3 GB

We are not going to do any transformations of the source data. Only minor tweakings of table encodings were applied:

```
CREATE TABLE IF NOT EXISTS tripdata (
  pickup_date Date DEFAULT toDate(pickup_datetime) CODEC(Delta, LZ4),
  id UInt64,
  vendor_id String,
  pickup_datetime DateTime CODEC(Delta, LZ4),
  dropoff_datetime DateTime,
  passenger_count UInt8,
  trip_distance Float32,
  pickup_longitude Float32,
  pickup_latitude Float32,
  rate_code_id String,
  store_and_fwd_flag String,
  dropoff_longitude Float32,
  dropoff_latitude Float32,
  payment_type LowCardinality(String),
  fare_amount Float32,
  extra String,
  mta_tax Float32,
  tip_amount Float32,
  tolls_amount Float32,
  improvement_surcharge Float32,
  total_amount Float32,
  pickup_location_id UInt16,
  dropoff_location_id UInt16,
  junk1 String,
  junk2 String)
ENGINE = MergeTree
PARTITION BY toYYYYMM(pickup_date)
ORDER BY (vendor_id, pickup_location_id, pickup_datetime);
```

Once the table is created, data can be loaded with a single SQL statement as:

```
set max_insert_threads=32;

INSERT INTO tripdata
SELECT *
```

```
FROM s3('https://<bucket_name>/nyc_taxi_rides/data/tripdata/data-20*.csv.gz',
'CSVWithNames',
'pickup_date Date, id UInt64, vendor_id String, pickup_datetime DateTime, dropoff_da
```

Once the table is created, data can be loaded with a single SQL statement as:

The syntax is not standard; we use the ClickHouse table function ‘s3()’ in order to connect to and read from an S3 bucket. ClickHouse table functions are a powerful extension technique that allows to integrate a DBMS with external data sources without changing the SQL syntax. ClickHouse has a bunch of those, and the ‘s3()’ table function is a welcome addition.

copy

```
0 rows in set. Elapsed: 280.696 sec. Processed 1.31 billion rows, 167.39 GB (4.67 mi
```

It takes less than 5 minutes in order to load 1.3 billion rows from the S3 bucket! Note that wildcards are used in order to load multiple files, and Clickhouse can process gzipped data natively as well!

In the same way we can load the ‘taxi_zones’ table — the table is small so loading is almost instantaneous from S3.

copy

```
CREATE TABLE IF NOT EXISTS taxi_zones (
  location_id UInt16,
  zone String,
  create_date Date DEFAULT toDate(0)
)
ENGINE = MergeTree
ORDER BY (location_id);

INSERT INTO taxi_zones
SELECT *
FROM s3('https://<bucket_name>/nyc_taxi_rides/data/taxi_zones/data-*.csv.gz',
'CSVWithNames', 'location_id UInt16, zone String, create_date Date', 'gzip');
```

Once the data is loaded, it is a good practice to inspect table sizes:

copy

```
SELECT
    table,
    sum(rows),
```

```
    sum(data_uncompressed_bytes) AS uc,
    sum(data_compressed_bytes) AS c,
    uc / c AS ratio
FROM system.parts
WHERE (database = 'default') AND active
GROUP BY table

┌─table──────────────┬─€sum(rows)─┬──────────€uc─┬──────────€c─┬──────────────────ratio─┐
│ tripdata           │ 1310903963 │ 104469253248 │ 37563206521 │ 2.7811591778671945 │
│ taxi_zones         │        263 │         5495 │        3697 │ 1.4863402758993778 │
└────────────────────┴────────────┴──────────────┴─────────────┴────────────────────────┘
```

As you can see, uncompressed data size is above 100GB, which lands in ClickHouse as 35GB for the main table with 2.8 times compression ratio. We usually expect ClickHouse to compress more aggressively, but the table has a lot of random floats that are hard to pack effectively.

The data loading process was very fast and convenient and it took us less than 10 minutes to get ready for queries.

Queries

Following Mark Litwintschik examples we used several simple queries in order to benchmark performance.

Q1. Group by a single column.

copy

```
SELECT
    passenger_count,
    avg(total_amount)
FROM tripdata
GROUP BY passenger_count
```

Q2. Group by two columns.

copy

```
SELECT
    passenger_count,
    toYear(pickup_date) AS year,
    count(*)
FROM tripdata
```

```
GROUP BY passenger_count, year
```

Q3. Group by three columns.

copy

```
SELECT
    passenger_count,
    toYear(pickup_date) AS year,
    round(trip_distance) AS distance,
    count(*)
FROM tripdata
GROUP BY passenger_count, year, distance
ORDER BY year, count(*) DESC
```

We have added two more queries to check joins.

Q4. Query with a JOIN to taxi_zones table

copy

```
SELECT
    tz.zone AS zone,
    count() AS c
FROM tripdata AS td
LEFT JOIN taxi_zones AS tz ON td.pickup_location_id = tz.location_id
GROUP BY zone
ORDER BY c DESC
```

Q5. Where condition on the joined table.

copy

```
SELECT count(*)
FROM tripdata AS td
INNER JOIN taxi_zones AS tz ON td.pickup_location_id = tz.location_id
WHERE tz.zone = 'Midtown East'
```

Here are the results (all numbers — query time in seconds).

Query	**ClickHouse** **m5.8xlarge**

Data load	280
Q1	0.62
Q2	1.11
Q3	1.78
Q4	0.94
Q5	0.33

Numbers do not look bad for a 1.3B rows dataset, but let’s look at comparisons.

Redshift

Now we repeat the same experience with Redshift. Redshift has a limited number of options for instance types to select from, the closest to m5.8xlarge instances we were using for ClickHouse is Redshift dc2.8xlarge instance. dc2.8xlarge is equipped with 32 vCPUs, 244GB of RAM and 2.5TB local SSD. It is important to note that Redshift forces users to use at least two dc2.8xlarge nodes per cluster, which raises the cost of the cluster to $230.40 per day.

The data loading is easy using standard SQL COPY statement:

```
COPY tripdata
FROM 's3://<bucket_name>/nyc_taxi_rides/data/tripdata/'
CREDENTIALS ‘’
DELIMITER ','
  EMPTYASNULL
  ESCAPE
  GZIP
  MAXERROR 100000
  REMOVEQUOTES
  TRIMBLANKS
  IGNOREHEADER
  TRUNCATECOLUMNS;

[2020-07-28 19:43:57] completed in 8 m 26 s 624 ms
```

This is very fast but it takes 80% more time compared to ClickHouse.

We run the same queries on Redshift using psql with query result cache disabled. Here are the results we’ve got:

Query	ClickHouse m5.8xlarge	RedShift dc2.8xlarge x2
Data load	280	506
Q1	0.62	0.59
Q2	1.11	0.82
Q3	1.78	2.8
Q4	0.94	0.64
Q5	0.33	0.45

As you can see, the query performance is close between the two databases. ClickHouse is slower on some queries, and faster on others. The total query time is lower with ClickHouse, but it is fair to say there is a tie here.

Let's note, however, that Redshift is running on two nodes and may distribute data and query execution accordingly. For a fair comparison we need to add one more node to ClickHouse as well.

Scaling ClickHouse Out

Scaling from one to two servers requires some configuration and schema changes. Please refer to our webinar for the details of ClickHouse clustering: "Strength in Numbers: Introduction to ClickHouse cluster performance". Since we run ClickHouse inside Kubernetes, we can use Altinity clickhouse-operator for Kubernetes that turns adding a node to the cluster to a one click job.

When a new node is added to the ClickHouse cluster, data is not redistributed automatically. So there are two options:

1. Reload data from S3 to the distributed table.
2. Reload data from the local to the distributed table with a simple INSERT SELECT statement.

We tried the first approach in order to measure load time into a distributed table.

```
CREATE TABLE tripdata_local ON CLUSTER '{cluster}' AS tripdata;

CREATE TABLE tripdata_d ON CLUSTER '{cluster}' AS tripdata Engine = Distributed('{cl
```

```
INSERT INTO tripdata_d SELECT * FROM s3(...);
```

Unfortunately, we hit a problem in ClickHouse at this point. The loading into the distributed table was 3-4 times slower due to lack of parallelisation when processing an insert. This is not acceptable by ClickHouse standards, and a fix has been already submitted. So in order to speed things up until the new ClickHouse version is available, we made a trick and re-distributed the table manually using the following technique:

```
INSERT INTO tripdata_local SELECT *
FROM tripdata
WHERE (cityHash64(*) % 2) = 0

0 rows in set. Elapsed: 84.095 sec. Processed 1.31 billion rows, 167.40 GB (15.59 mi

INSERT INTO FUNCTION remote('second_node_address', default.tripdata_local) SELECT *
FROM tripdata
WHERE (cityHash64(*) % 2) = 1

0 rows in set. Elapsed: 335.122 sec. Processed 1.31 billion rows, 167.40 GB (3.91 mi
```

Here we used yet another table function ‘remote()’ that allows us to query between ClickHouse nodes. Note that we inserted data into a function, which is also a unique ClickHouse extension.

Below are query results for all tested configurations. We have also added Mark Litwintschick’s historical data in the last two columns for the reference.

Query	**ClickHouse m5.8xlarge, Aug 2020**	**ClickHouse m5.8xlarge x2, Aug 2020**	**RedShift dc2.8xlarge x2, Aug 2020**	**ClickHouse c5d.9xlarge x3, Jan 2019**	**Redshift ds2.8xlarge x6, June 2016**
Data load	280	n/a	506	n/a	673
Q1	0.62	0.35	0.59	0.69	1.25
Q2	1.11	0.58	0.82	0.58	2.25
Q3	1.78	0.94	2.8	0.98	2.97

Q4	0.94	0.48	0.64		
Q5	0.33	0.19	0.45		
Cost per day	**$36.86***	**$73.73***	**$230.4**	**$124.4**	**$1094**

** Cost for ClickHouse does not include storage $3/TB/day.*

As expected, adding an extra node to ClickHouse has improved query performance almost twice delivering constantly better response time. The superior ClickHouse performance comes at â…“ of the Redshift cost.

Another Look at Q5

Looking back at Q5, in a real ClickHouse application we would not write the query this way. ClickHouse does not push the join condition properly as a filter to the main table. There is a task to fix this. We would rewrite it as follows:

copy

```
SELECT count(*)
FROM tripdata AS td
WHERE pickup_location_id IN
(
    SELECT location_id
    FROM taxi_zones
    WHERE zone = 'Midtown East'
)
```

Such a change reduces the query time from 0.33s to almost instant 0.008s — it only needs to count 2M rows. ‘pickup_location_id’ is a part of the primary key, so there is no surprise. But if we change it to ‘dropoff_location_id’, that is not a part of the primary key, the performance is still under 0.1s. Strangely enough, if we try the same on Redshift the query time increases from 0.45s to 0.54s.

This is an example of the “ClickHouse way” of doing efficient applications. The proper use of technology can give substantially better results than pure apples-to-apples SQL benchmarks. We will be getting back to that in our blog in the future.

Conclusion

Compared to Mark’s benchmark years ago, the 2020 versions of both ClickHouse and Redshift show much better performance. It is good to see that both products have improved over time. The Redshift progress is remarkable, thanks to new dc2 node types and a lot of engineering work to speed things up. However, open source ClickHouse continues to outperform Redshift on similarly sized hardware, and the difference increases as the query complexity grows.

It is not just performance that makes ClickHouse unique. The open source development model makes it evolve driven by user needs and specific use cases. In the next article we will focus on some specific scenarios where ClickHouse SQL features help to solve problems that are hard to address effectively in other systems. Stay tuned!

#benchmark #ClickHouse #RedShift

← PREVIOUS
Machine Learning Models as Tables

NEXT →
Introduction to ClickHouse® Backups and Altinity Backup for ClickHouse®

One Comment

Andrei says:
25th May 2021 at 11:46 pm

Thank you. Great articles with lots of details.

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

Save my name, email, and website in this browser for the next time I comment.

Post Comment